Revision Game Cards — 2.2 Problem Solving & Programming

Print and cut along the borders. ✂️ Loop cards: each player reads the bottom "Who has…?"; whoever holds the matching "I have…" reads it out, then their own clue — the chain visits every card and loops back to the start.

Loop cards (dominoes) — cut out & deal

1 / 16

I HAVE

Recursion

WHO HAS…

Who has the condition that stops a recursive subroutine??

2 / 16

I HAVE

Base case

WHO HAS…

Who has the error caused when the stopping condition is missing or never reached??

3 / 16

I HAVE

Stack overflow

WHO HAS…

Who has the structure onto which too many frames are pushed??

4 / 16

I HAVE

Call stack

WHO HAS…

Who has the region of visibility where a variable can be accessed??

5 / 16

I HAVE

Scope

WHO HAS…

Who has the variable visible only inside the subroutine that declares it??

6 / 16

I HAVE

Local variable

WHO HAS…

Who has the variable visible throughout the whole program??

7 / 16

I HAVE

Global variable

WHO HAS...

Who has a subroutine that performs a task and returns a single value??

8 / 16

I HAVE

Function

WHO HAS...

Who has a subroutine that performs a task but returns no value??

9 / 16

I HAVE

Procedure

WHO HAS...

Who has passing a copy of an argument so the original is unchanged??

10 / 16

I HAVE

By value

WHO HAS...

Who has passing the memory address so the original can be changed??

11 / 16

I HAVE

By reference

WHO HAS...

Who has the OOP principle of bundling data and methods while keeping attributes private??

12 / 16

I HAVE

Encapsulation

WHO HAS...

Who has the OOP principle of a subclass acquiring a superclass's attributes and methods??

13 / 16

I HAVE

Inheritance

WHO HAS...

Who has the OOP principle of the same method call behaving differently for different object types??

14 / 16

I HAVE

Polymorphism

WHO HAS...

Who has building a solution step by step and returning to the last choice on a dead end??

I HAVE

Backtracking

WHO HAS...

Who has a rule of thumb giving a good-enough but not optimal answer quickly??

I HAVE

Heuristic

WHO HAS...

Who has a subroutine that calls itself, needing a base case and a recursive case??

Taboo cards — describe the term without the banned words

Recursion

Don't say:

- itself
- base case
- calls
- stack

Encapsulation

Don't say:

- private
- data
- methods
- hiding

Inheritance

Don't say:

- subclass
- parent
- acquires
- superclass

By reference

Don't say:

- address
- copy
- original
- memory

Heuristic

Don't say:

- rule of thumb
- optimal
- good-enough
- quick

Local variable

Don't say:

- subroutine
- scope
- global
- inside

Polymorphism

Don't say:

- method
- override
- different
- object

Backtracking

Don't say:

- dead end
- undo
- maze
- try

Bingo — word bank & ready-to-cut cards

Word bank (students fill a blank 3×3 grid, or use the pre-filled cards below): Sequence, Selection, Count-controlled iteration, Condition-controlled iteration, Recursion, Base case, Call stack, Stack overflow, Scope, Local variable, Global variable, Function, Procedure, By value, By reference, Class, Object, Encapsulation, Inheritance, Heuristic

BINGO 1

By value	Condition-controlled iteration	Procedure
Function	Count-controlled iteration	Stack overflow
Class	Scope	Call stack

BINGO 2

Selection	Call stack	Count-controlled iteration
Procedure	Base case	Inheritance
Function	Class	Local variable

BINGO 3

Object	By value	Stack overflow
Recursion	Heuristic	Class
Encapsulation	Count-controlled iteration	Global variable

BINGO 4

Class	Encapsulation	Function
Stack overflow	By reference	Object
Scope	Heuristic	Call stack

BINGO 5

By value	Condition-controlled iteration	Encapsulation
Object	Call stack	Procedure
Heuristic	By reference	Inheritance

BINGO 6

Class	Procedure	Encapsulation
By value	Local variable	Base case
Scope	Global variable	Count-controlled iteration

Bingo — caller's list (teacher: read the clue, students cross off the term)

#	Clue to read aloud	Answer
1	Instructions executed one after another, in order.	Sequence
2	Choosing between paths based on a condition (if / switch).	Selection
3	A loop repeating a fixed, known number of times (for...next).	Count-controlled iteration
4	A loop repeating until a condition changes (while / do...until).	Condition-controlled iteration
5	A subroutine that calls itself, with a base case and a recursive case.	Recursion
6	The condition that stops recursion, with no further recursive call.	Base case
7	The structure of frames holding parameters, locals and return addresses for active calls.	Call stack
8	The error from too many pushed frames, e.g. a missing base case.	Stack overflow
9	The region of a program where a variable can be accessed.	Scope
10	A variable accessible only within the subroutine that declares it.	Local variable
11	A variable accessible throughout the whole program.	Global variable
12	A subroutine that performs a task and returns a single value.	Function
13	A subroutine that performs a task but returns no value.	Procedure
14	Passing a copy of an argument, so the original is unaffected.	By value
15	A template or blueprint defining the attributes and methods of its objects.	Class
16	A specific instance of a class, created at run time.	Object

Quick-fire — clue & answer (self-test / quiz-quiz-trade)

#	Clue	Answer
1	A while loop tests its condition here, so the body may run zero times.	At the start (pre-conditioned)
2	A do...until loop guarantees the body runs at least this many times.	Once (post-conditioned)
3	Each recursive call pushes one of these onto the call stack.	A stack frame
4	The variable in a subroutine definition that receives an argument.	Parameter
5	The actual value supplied at the point of the call.	Argument
6	A local of the same name does this to a global inside its subroutine.	Shadows it
7	OCR's keyword for the constructor method that runs on object creation.	new
8	A subclass replacing an inherited method with the same name and parameters.	Overriding
9	Same method name but different parameter lists.	Overloading
10	Searching large data sets for hidden patterns, e.g. basket analysis.	Data mining
11	Splitting a process into stages so different items are processed at once.	Pipelining
12	Repeatedly splitting a problem into smaller instances, solving and combining.	Divide and conquer